Student Management System - MySQL Capstone Project

Project Objective

Design and implement a Student Management System using MySQL to manage student details, departments, faculty, courses, course enrollments, examinations and marks. The system should maintain accurate academic records and generate useful reports using SQL queries.

Problem Statement

A college wants to digitize its student information, department details, faculty records, course registrations and examination results. Currently, maintaining these records manually makes it difficult to track student information, monitor course enrollments and analyze academic performance.

Design a relational database to efficiently manage student academic records, maintain data consistency and generate meaningful reports for college administration.

MySQL Workflow

- Requirement Analysis
- Identify Entities
- ER Diagram
- Normalization (1NF → 3NF)
- Create Database
- Create Tables
- Insert Sample Data
- CRUD Operations
- Numeric, String & Date Functions
- GROUP BY & HAVING
- Joins & Subqueries
- Views
- Transactions
- Stored Procedures & Triggers

Database Design

- Departments (dept_id PK, dept_name, hod_name, location)
- Students (student_id PK, student_name, phone, email, city, admission_date, dept_id FK)
- Faculty (faculty_id PK, faculty_name, email, designation, hire_date, dept_id FK)
- Courses (course_id PK, course_name, credits, fee, dept_id FK)
- Enrollments (enrollment_id PK, student_id FK, course_id FK, enrollment_date, enrollment_status)
- Exams (exam_id PK, course_id FK, exam_name, exam_date, max_marks)

- Marks (mark_id PK, student_id FK, exam_id FK, marks_obtained)

Insert Data

INSERT INTO Departments VALUES (1, 'Computer Science', 'Dr. Meera Rao', 'Block A'), (2, 'Data Science', 'Dr. Arun Kumar', 'Block B'), (3, 'Electronics', 'Dr. S. Reddy', 'Block C'), (4, 'Commerce', 'Dr. Lakshmi', 'Block D');

INSERT INTO Students VALUES

(101, 'Aarav Sharma', '9876500101', 'aarav@mail.com', 'Vijayawada', '2023-06-15', 1),

(102, 'Bhavya Reddy', '9876500102', 'bhavya@mail.com', 'Guntur', '2022-06-18', 2),

(103, 'Charan Kumar', '9876500103', 'charan@mail.com', 'Vijayawada', '2023-06-15', 1),

(104, 'Divya Rao', '9876500104', 'divya@mail.com', 'Hyderabad', '2022-06-18', 2),

(105, 'Farhan Ali', '9876500105', 'farhan@mail.com', 'Nellore', '2023-06-20', 3),

(106, 'Geetha Nair', '9876500106', 'geetha@mail.com', 'Vizag', '2022-06-18', 1),

(107, 'Harish Varma', '9876500107', 'harish@mail.com', 'Guntur', '2023-06-20', 4),

(108, 'Ishita Singh', '9876500108', 'ishita@mail.com', 'Vijayawada', '2022-06-18', 2),

(109, 'Kiran Teja', '9876500109', 'kiran@mail.com', 'Ongole', '2023-06-20', 3),

(110, 'Lavanya Devi', '9876500110', 'lavanya@mail.com', 'Hyderabad', '2022-06-18', 4);

INSERT INTO Faculty VALUES

(201, 'Ananya Rao', 'ananya@college.edu', 'Professor', '2018-07-01', 1), (202, 'Rahul Varma', 'rahul@college.edu', 'Assistant Professor', '2020-06-15', 1), (203, 'Sowmya Iyer', 'sowmya@college.edu', 'Associate Professor', '2017-08-10', 2), (204, 'Vikram Reddy', 'vikram@college.edu', 'Assistant Professor', '2021-01-05', 3), (205, 'Priya Das', 'priya@college.edu', 'Professor', '2016-06-20', 4);

INSERT INTO Courses VALUES (301, 'Database Management Systems', 4, 45000, 1), (302, 'Python Programming', 4, 42000, 2), (303, 'Computer Networks', 3, 38000, 1), (304, 'Machine Learning', 4, 52000, 2), (305, 'Digital Electronics', 3, 40000, 3), (306, 'Business Statistics', 3, 35000, 4), (307, 'Web Technologies', 3, 39000, 1),(308, 'Data Visualization', 3, 41000, 2);

INSERT INTO Enrollments VALUES (501, 101, 301, '2026-06-20', 'Enrolled'), (502, 103, 301, '2026-06-20', 'Enrolled'), (503, 106, 301, '2026-06-21', 'Enrolled'), (504, 110, 303, '2026-06-21', 'Enrolled'), (505, 102, 302, '2026-06-20', 'Enrolled'), (506, 104, 302, '2026-06-20', 'Enrolled'), (507, 108, 304, '2026-06-21', 'Enrolled'), (508, 108, 304, '2026-06-21', 'Enrolled'), (509, 101, 303, '2026-06-22', 'Enrolled'), (510, 103, 307, '2026-06-22', 'Enrolled'), (511, 105, 305, '2026-06-22', 'Enrolled'), (512, 109, 305, '2026-06-22', 'Enrolled'), (513, 102, 304, '2026-06-23', 'Enrolled'), (514, 104, 308, '2026-06-23', 'Enrolled'), (515, 108, 302, '2026-06-23', 'Enrolled'), (516, 107, 306, '2026-06-23', 'Enrolled'), (517, 110, 306, '2026-06-23', 'Enrolled'), (518, 105, 306, '2026-06-24', 'Withdrawn'), (519, 101, 307, '2026-06-24', 'Enrolled'), (520, 103, 301, '2026-06-24', 'Completed'), (521, 107, 308, '2026-06-24', 'Enrolled');

INSERT INTO Exams VALUES (401, 301, 'Internal Assessment 1', '2026-08-05', 30), (402, 301, 'Internal Assessment 2', '2026-09-05', 30), (403, 302, 'Internal Assessment 1', '2026-08-06', 30), (404, 303, 'Mid-Term Examination', '2026-08-07', 50), (405, 304, 'Mid-Term Examination', '2026-08-08', 50), (406, 305, 'Mid-Term Examination', '2026-08-09', 50), (407, 306, 'Mid-Term Examination', '2026-08-10', 50), (408, 307, 'Final Examination', '2026-10-05', 100), (409, 308, 'Final Examination', '2026-10-06', 100);

INSERT INTO Marks VALUES (601, 101, 401, 27), (602, 103, 401, 21), (603, 106, 401, 18), (604, 101, 402, 28), (605, 103, 402, 23), (606, 106, 402, 20), (607, 102, 403, 24), (608, 104, 403, 19), (609, 108, 403, 28), (610, 101, 404, 42), (611, 103, 404, 37), (612, 102, 405, 44), (613, 108, 405, 39), (614, 105, 406, 36), (615, 109, 406, 41), (616, 107, 407, 38), (617, 110, 407, 45), (618, 101, 408, 86), (619, 103, 408, 78), (620, 104, 409, 88), (621, 102, 409, 82);

Business Reports

1. Display all student details from the Students table.
2. Display student names, phone numbers and email addresses of students belonging to department ID 1.
3. Retrieve the distinct cities from which students have been admitted.
4. Find students who belong to department ID 1 or department ID 2 using the IN operator.
5. Display courses having fees between ₹38,000 and ₹45,000.
6. Find students whose names start with the letter 'A' or end with the letter 'a' using pattern matching.
7. Find the total number of students in each department.
8. Calculate the average, highest and lowest course fees for each department.
9. Calculate the total marks and average marks obtained by each student across all examinations.

10. Display the ceiling and floor values of the average marks obtained by each student.
11. Calculate the percentage obtained by each student in every examination and round the result to two decimal places.
12. Display student names in uppercase and their email addresses in lowercase.
13. Generate a student identification display by concatenating the student name and student ID. Example: Aarav Sharma - 101.
14. Display student names and their completed years of study in the college using an appropriate date function.
15. Display examination dates in the format DD-Month-YYYY along with the month name.
16. Display a combined list of student names and faculty names using UNION.
17. Display student IDs who have enrolled in courses and student IDs who have obtained examination marks using UNION ALL.
18. Find student IDs who are present in both the Students and Enrollments tables using INTERSECT.
19. Find student IDs who have not enrolled in any course using EXCEPT.
20. Display student names along with their department names.
21. Display course names, department names and course fees.
22. Display faculty names, their designations and the departments in which they work.
23. Display student names, course names, enrollment dates and enrollment statuses.
24. Generate an examination result report displaying student name, course name, exam name, marks obtained and maximum marks.
25. Find students who have not enrolled in any course using NOT EXISTS.
26. Find courses whose fees are greater than the average fee of all courses.
27. Display students who scored more than the overall average marks.
28. Find the student or students who obtained the highest marks in each examination.
29. Display students who scored above the average marks of their respective examinations using a correlated subquery.
30. Create a view named Student_Course_Details to display student name, department name, course name, enrollment date and enrollment status.
31. Create a view named Student_Exam_Results to display student name, course name, examination name, marks obtained, maximum marks and result percentage.
32. Demonstrate COMMIT by inserting a new enrollment record and permanently saving the transaction.
33. Demonstrate ROLLBACK by updating a student's email address and reversing the change before committing.
34. Use SAVEPOINT while inserting two enrollment records. Roll back to the savepoint to cancel only the second insertion, then commit the first insertion.
35. Create a stored procedure that accepts a department ID as an IN parameter and displays all students belonging to that department.
36. Create a stored procedure that accepts a student ID as an input parameter and displays all courses enrolled by that student along with enrollment status.
37. Create a stored procedure that accepts an exam ID and returns the average marks obtained in that examination using an OUT parameter.

38. Create an AFTER DELETE trigger on the Students table to automatically store the details of a deleted student in a separate Students_Archive table.
39. Create an AFTER UPDATE trigger on the Marks table to store the old marks and updated marks of a student in a separate Marks_History table whenever marks are changed.
40. Create an AFTER INSERT trigger on the Enrollments table to automatically store the student ID, course ID, and enrollment date in a separate Enrollment_Log table whenever a new enrollment is created.